ACHILLES AGENT

Technical Security Report

Document Classification: Customer-Facing Technical Brief **Version:** 1.0 | **Date:** March 2026 | **Product:** ProjectAchilles Agent v0.6

1. Executive Summary

The Achilles Agent is a lightweight, purpose-built binary deployed to endpoints for continuous security validation. It executes authorized security tests, reports results, and self-updates — all under strict cryptographic controls. This document details the technical safeguards ensuring the agent operates securely, transparently, and with minimal footprint on customer infrastructure.

Key security properties:

- **Statically compiled Go binary** — zero runtime dependencies, no interpreter, no DLL/shared library attack surface
- **Machine-bound encrypted credentials** — API keys encrypted at rest with AES-256-GCM, bound to hardware identity
- **Ed25519-signed updates** — binary integrity verified cryptographically before any self-update
- **Zero-downtime key rotation** — dual-key grace period model enables seamless credential cycling
- **Immutable audit trail** — every lifecycle event logged in an append-only event store

2. Agent Architecture

2.1 Binary Properties

Property	Detail
Language	Go 1.24 (compiled, memory-safe)
Linking	Fully static (`CGO_ENABLED=0`)
Platforms	Windows (amd64), Linux (amd64), macOS (amd64 + arm64)
Size	~8–12 MB per platform
Symbols	Stripped (`-s -w` LDFLAGS) — no debug info in production binaries
Code Signing	Authenticode (Windows), ad-hoc codesign (macOS)

The agent has **no external dependencies** — no Python, .NET, Java, or shared libraries required. This eliminates an entire class of supply-chain and dependency-confusion attacks. Static compilation also means the exact binary that was built, signed, and verified is what runs on the endpoint.

2.2 Service Installation

The agent installs as a native system service with auto-restart on failure:

Platform	Service Manager	Restart Policy	Binary Permissions
Windows	SCM (`sc.exe`)	3 retries, 10s delay	SYSTEM + Administrators only (icacls)
Linux	systemd	`Restart=always`, 10s	`0700` (owner rwx)
macOS	launchd	`KeepAlive=true`	`0700` + ad-hoc codesign

Binary permissions are enforced at **three points**: installation, update, and every startup (retroactive hardening). On Windows, inherited ACLs are explicitly stripped, leaving only `SYSTEM(RX)` and `Administrators(F)`.

---

3. Enrollment & Authentication

3.1 One-Time Enrollment

Enrollment uses a **single-use, time-limited token** model:

1. Administrator generates an enrollment token via the management console
2. Token is **bcrypt-12 hashed** before storage — the plaintext is shown once and never persisted
3. Agent presents the token during enrollment over HTTPS
4. Server validates, registers the agent, and returns: Agent ID, API Key, Organization ID, Ed25519 update public key
5. Token use-count is atomically incremented (TOCTOU-safe) and cannot be reused beyond its limit

Anti-enumeration: Even when no valid tokens exist, the server performs a constant-time bcrypt comparison against a dummy hash, preventing timing-based token discovery.

3.2 Ongoing Authentication

Every agent request includes:

Header	Purpose
`Authorization: Bearer ak_<key>`	API key authentication
`X-Agent-ID`	Agent identity binding
`X-Agent-Version`	Version tracking
`X-Request-Timestamp`	Replay protection (±5 min clock skew tolerance)

The server always executes `bcrypt.compare()` regardless of whether the agent exists, ensuring **constant-time rejection** that reveals no information about registered agent IDs. Rate limiting is enforced at 100 requests per 15-minute window per agent.

3.3 API Key Rotation

The platform supports both **manual** and **automatic** key rotation with a zero-downtime dual-key model:

1. New API key generated and stored as "pending" alongside the active key
2. During a **5-minute grace period**, both keys authenticate successfully
3. New key delivered to the agent via its next heartbeat response
4. Agent persists the new key (encrypted) and begins using it immediately
5. Once the agent uses the new key, the old key is retired

Automatic rotation can be configured on a 30–365 day interval.

---

4. Communication Security

4.1 Transport Layer

- **TLS 1.2+** enforced for all remote communication (Go's default TLS stack)
- **HTTPS-only**: the agent rejects `http://` server URLs for non-localhost targets at startup
- **Redirect downgrade protection**: HTTP→HTTPS downgrades are blocked at enrollment and runtime
- **Custom CA support**: customers can provide their own CA certificate for internal PKI environments

4.2 Data in Transit

Data Flow	Content	Protection
Heartbeat (60s ± jitter)	Status, CPU/memory/disk metrics, version	TLS + API key auth
Task poll (30s ± jitter)	Task assignments	TLS + API key auth
Result submission	Exit code, stdout/stderr (1 MB cap), timing	TLS + API key auth
Binary download	Test binaries	TLS + SHA-256 integrity verification

All polling intervals include **±5s random jitter** to prevent synchronized bursts when many agents restart simultaneously.

---

5. Credential Protection at Rest

5.1 Machine-Bound Key Encryption

Agent API keys are **never stored in plaintext** on disk:

Component	Specification
KDF	PBKDF2-SHA256, 210,000 iterations (OWASP 2023)
Cipher	AES-256-GCM (authenticated encryption)
Salt	16 bytes, random per encryption
Nonce	12 bytes (GCM default), random per encryption
Binding	Machine hardware ID (platform-specific)

The encryption key is derived from a **platform-specific hardware identifier**:

Platform	Source	Property
Windows	Registry `MachineGuid`	Persistent across reboots
Linux	`/etc/machine-id` (systemd)	Unique per installation
macOS	`IOPlatformUUID`	Tied to hardware

Configuration files copied to a different machine cannot be decrypted. A stolen config file is unusable without the original hardware — the agent must re-enroll.

5.2 File Security

- **Config permissions:** `0600` (Unix) or `SYSTEM + Administrators` only (Windows)
- **Work directory:** `0700` — prevents world-readable test output
- **Legacy migration:** Plaintext keys are auto-encrypted on first load

---

6. Binary Integrity & Update Security

6.1 Code Signing

Platform	Method	Tool	Certificate
Windows	Authenticode	`osslsigncode`	Customer-managed PKCS#12
macOS	Ad-hoc signature	`rcodesign`	None required
Linux	—	—	Not applicable

Certificate passwords are **never passed as CLI arguments** — they use temporary files (`0600`) deleted immediately after signing.

6.2 Ed25519 Signed Updates

Self-updates are protected by a **detached Ed25519 signature scheme**:

1. Server computes SHA-256 hash of the new binary
2. Hash signed with Ed25519 private key (auto-generated, `0600` permissions)
3. Agent receives the public key during enrollment
4. Before applying any update, the agent **verifies the signature** against the binary's hash
5. Verification failure **blocks the update entirely**

> This provides defense-in-depth: even if TLS were compromised, a man-in-the-middle cannot inject a malicious binary without the Ed25519 private key.

6.3 Update Application

Platform	Strategy	Safety
Windows	Rename current → `.old`, swap new in	Rollback on failure via `.old` restore
Linux	Atomic POSIX rename	Atomic — no partial state
macOS	Atomic rename + re-codesign	Satisfies Launch Constraints

The service manager automatically restarts the agent after update. Windows includes a **Task Scheduler failsafe** if SCM restart does not trigger.

---

7. Task Execution & Process Isolation

7.1 Execution Model

1. Agent polls → server assigns **one task at a time** (no parallel execution)
2. Binary downloaded with **SHA-256 and file-size verification**
3. Placed in isolated temp directory, permissions set to `0700`
4. Executed with configurable timeout (default 5 minutes)
5. stdout/stderr captured (1 MB cap) and reported with exit code

7.2 Process Management

Platform	Isolation Mechanism	Cleanup Guarantee
Windows	**Job Object** (`KILL_ON_JOB_CLOSE`)	All child processes terminated atomically
Linux/macOS	Context-based timeout	`SIGTERM` → 10s grace → `SIGKILL`

The Windows Job Object ensures **no orphaned child processes** survive after a test completes or times out.

---

8. Monitoring & Audit Trail

8.1 Heartbeat Telemetry

Every 60 seconds (± jitter), the agent reports: operational status, system metrics (CPU, memory, disk, uptime), hostname, OS, architecture, and current version. The server uses heartbeats for **offline detection** (3-minute threshold), version tracking, and fleet health dashboards. Heartbeat history is retained for 30 days.

8.2 Immutable Event Log

All lifecycle events are recorded in an **append-only audit table**:

Event Type	Trigger
`enrolled`	Agent completed enrollment
`came_online` / `went_offline`	Heartbeat gap detection
`task_completed` / `task_failed`	Test execution outcome
`version_updated`	Binary self-update applied
`key_rotated`	API key rotation completed
`status_changed` / `decommissioned`	Admin action on agent

8.3 Remote Uninstall

Agents can be remotely uninstalled with full traceability:

1. Agent receives uninstall task and reports acknowledgment while still authenticated
2. Platform-specific cleanup: service removal and optional file deletion
3. Agent exits without restart — event logged permanently

9. Security Audit Results

The agent underwent a dedicated security review. **All findings rated Medium or above have been remediated:**

Finding	Severity	Status
TLS verification bypass without safeguard	High	**Remediated**
No API key rotation mechanism	High	**Remediated**
No replay attack protection	Medium-High	**Remediated**
Enrollment token timing oracle	Medium	**Remediated**
Unsigned agent updates	Medium	**Remediated**
Rate limiting gaps on sensitive endpoints	Medium	**Remediated**
Plaintext credential storage on disk	Medium	**Remediated**
Overly permissive file permissions	Medium	**Remediated**

10. Security Controls Summary

Layer	Control
Binary	Static Go, zero dependencies, stripped symbols, code-signed
Transport	TLS 1.2+, HTTPS enforcement, redirect downgrade protection
Authentication	Bcrypt-12 hashed keys, constant-time comparison, replay protection
Credentials at Rest	AES-256-GCM + PBKDF2 (210K iter.), hardware-bound
Key Lifecycle	Auto-rotation (30–365 day), zero-downtime dual-key model
Update Integrity	SHA-256 + Ed25519 signature, atomic binary swap
Process Isolation	Windows Job Objects, configurable timeouts, graceful shutdown
File System	`0700` / `0600` (Unix), SYSTEM+Admins ACL (Windows)
Audit	Append-only event log, 30-day heartbeat history
Fleet Ops	Remote uninstall, offline detection, stale task cleanup

For questions about the Achilles Agent security architecture, contact your ProjectAchilles account representative.